16. E2E 테스트 매니페스트 및 네트워크 구성 분석

개요

Omni의 E2E 테스트 프레임워크는 Docker Compose 기반의 테스트넷을 구성하여 전체 네트워크의 통합 테스트를 수행합니다. 이 시스템은 매니페스트 파일을 통해 네트워크 토폴로지를 정의하고, 인프라 프로바이더를 통해 실제 테스트 환경을 배포합니다.

매니페스트 시스템

매니페스트 파일 구조

매니페스트 파일은 TOML 형식으로 작성되며, 테스트 네트워크의 전체 구성을 정의합니다:

```
# devnet0.toml 예시

network = "devnet"

anvil_chains = ["mock_op", "mock_arb"]

ephemeral_genesis = "2_magellan"



[node.validator01]



[node.fullnode01]

mode="archive"
```

주요 매니페스트 타입

1. Devnet 매니페스트

- **devnet0.toml**: 최소 구성 (단일 검증자, 아카이브 노드, 2개 anvil 체인)
- **devnet1.toml**: 표준 개발 환경
- **devnet2.toml**: 확장된 개발 환경
- **devnetsolve.toml**: SolverNet 테스트용

2. CI 매니페스트

```
network = "devnet"

anvil_chains = ["mock_l2", "mock_l1"]

multi_omni_evms = true

pingpong_n = 5

evidence = 1

all_e2e_tests = true


[node.validator01]

[node.validator02]

[node.validator03]

[node.validator04]

start_at = 20

state_sync = true


[validator_update.10]

fullnode01 = 100 # 높이 10에서 검증자 업데이트
```

```
[perturb]

validator02_evm = ["stopstart"]

relayer = ["restart"]

validator03 = ["rollback"]
```

3. 프로덕션 매니페스트

- **staging.toml**: 스테이징 환경
- **mainnet.toml**: 메인넷 환경
- **omega.toml**: 오메가 테스트넷

매니페스트 데이터 구조

```go
type Manifest struct {

e2e.Manifest

Network netconf.ID `toml:"network"`

// 체인 구성

AnvilChains []string  `toml:"anvil_chains"`

PublicChains []string  `toml:"public_chains"`

SVMChains []string  `toml:"svm_chains"`

// EVM 구성

MultiOmniEVMs  bool  `toml:"multi_omni_evms"`

// 테스트 구성
```

```
PingPongN uint64 `toml:"pingpong_n"`

AllE2ETests bool `toml:"all_e2e_tests"`

// 네트워크 설정

Forks map[string]string

Keys map[string]map[key.Type]string `toml:"keys"`

// 장애 시뮬레이션

Perturb map[string][]Perturb `json:"perturb"`

// 이미지 태그 고정

PinnedHaloTag string `toml:"pinned_halo_tag"`

PinnedMonitorTag string `toml:"pinned_monitor_tag"`

PinnedRelayerTag string `toml:"pinned_relayer_tag"`

PinnedSolverTag string `toml:"pinned_solver_tag"`

// 네트워크 업그레이드

EphemeralGenesis string `toml:"ephemeral_genesis"`

// 기능 플래그

FeatureFlags feature.Flags `toml:"feature_flags"`

}
```

Docker Compose 구성

컨테이너 오케스트레이션

E2E 프레임워크는 `compose.yaml.tmpl` 템플릿을 사용하여 Docker Compose 파일을 동적으로 생성합니다:

```
version: '2.4'

networks:

{{ .NetworkName }}:

labels:

e2e: true

driver: bridge

{{- if .Network }}

ipam:

driver: default

config:

- subnet: {{ .NetworkCIDR }}

{{- end }}



services:

{{- range .Nodes }}

{{ .Name }}:

labels:

e2e: true

container_name: {{ .Name }}

image: {{ .Version }}

restart: unless-stopped

init: true
```

```
ports:
- {{ if $.BindAll }}26656:{{end}}26656  # CometBFT P2P
- {{ if .ProxyPort }}{{ .ProxyPort }}:{{ end }}26657  # CometBFT RPC
- {{ if $.BindAll }}9999:{{end}}9999  # Cosmos gRPC
- {{ if $.BindAll }}1317:{{end}}1317  # Cosmos REST
- {{ if $.BindAll }}8545:{{end}}8545  # EVM JSON-RPC
volumes:
- ./{{ .Name }}:/halo
networks:
{{ $.NetworkName }}:
{{ if $.Network }}ipv4_address: {{ .InternalIP }}{{ end }}
{{end}}
```

서비스 타입별 구성

1. Halo 노드 (합의 클라이언트)

- **포트**: 26656 (P2P), 26657 (RPC), 9999 (gRPC), 1317 (REST), 8545 (EVM)
- **볼륨**: `./node_name:/halo`
- **환경변수**: 커스텀 제네시스 설정

2. Anvil 체인 (모의 EVM 체인)

```
{{ .Chain.Name }}:
container_name: {{ .Chain.Name }}
image: omniops/anvilproxy:{{or $.AnvilProxyTag "main"}}
```

```
environment:
- ANVILPROXY_CHAIN_ID={{ .Chain.ChainID }}
- ANVILPROXY_BLOCK_TIME={{.Chain.BlockPeriod.Seconds}}
- ANVILPROXY_SLOTS_IN_AN_EPOCH=4
{{ if .ForkRPC }}- ANVILPROXY_FORK_URL={{ .ForkRPC }}{{ end }}
ports:
- {{ if .ProxyPort }}{{ .ProxyPort }}:{{ end }}8545
```

3. Omni EVM (Geth 기반)

```
{{ .InstanceName }}:
container_name: {{ .InstanceName }}
image: ethereum/client-go:{{ $.InitialGethTag $i }}
command:
- --config=/geth/config.toml
- --nat=extip:{{ .AdvertisedIP }}
- --pprof
- --metrics
- --graphql
{{ if .IsArchive }}- --gcmode=archive{{ end }}
ports:
- {{ if $.BindAll }}8551:{{end}}8551  # Auth RPC
- {{ if .ProxyPort }}{{ .ProxyPort }}:{{ end }}8545  # HTTP RPC
```

```
- {{ if $.BindAll }}30303:{{end}}30303  # P2P
volumes:
- ./{{ .InstanceName }}:/geth
```

4. 보조 서비스

- **Relayer**: 크로스체인 메시지 릴레이
- **Monitor**: 네트워크 모니터링
- **Solver**: SolverNet 참조 구현
- **Prometheus**: 메트릭 수집

네트워크 모드

1. Docker 모드 (로컬 개발)

- 모든 서비스가 단일 Docker 네트워크에서 실행
- `BindAll=true` 로 모든 포트를 호스트에 바인딩
- 고정 IP 주소 할당 없음

2. VM Compose 모드 (분산 테스트)

- 여러 VM에 서비스 분산 배포
- 고정 IP 주소 할당
- GCP 기반 인프라 관리

인프라 프로바이더 시스템

Provider 인터페이스

```go
type InfraProvider interface {
Setup() error
StartNodes(ctx context.Context, nodes ...*e2e.Node) error
Restart(ctx context.Context, cfg ServiceConfig) error
Upgrade(ctx context.Context, cfg ServiceConfig) error
Clean(ctx context.Context) error
StopTestnet(ctx context.Context) error
GetInfrastructureData() *e2e.InfrastructureData
}
```

Docker Provider

로컬 개발 환경을 위한 Docker Compose 기반 프로바이더:

```go
func (p *Provider) Setup() error {
// Docker Compose 파일 생성
compose, err := docker.GenerateComposeFile(def)
if err != nil {
return errors.Wrap(err, "generate compose file")
}
// EVM 초기화 스크립트 생성
evmInit, err := docker.GenerateOmniEVMInitFile(def)
if err != nil {
return errors.Wrap(err, "generate evm init file")
```

```
}
return nil
}
```

VMCompose Provider

분산 VM 환경을 위한 프로바이더:

```
func (p *Provider) Setup() error {
// VM별로 서비스 그룹화
for vmIP, services := range groupByVM(p.Data.Instances) {
// VM별 Docker Compose 파일 생성
compose, err := docker.GenerateComposeFile(def)
if err != nil {
return errors.Wrap(err, "generate compose file")
}
// VM별 파일 저장
err = os.WriteFile(filepath.Join(p.Testnet.Dir,
vmComposeFile(vmIP)), compose, 0o644)
if err != nil {
return errors.Wrap(err, "write compose file")
}
}
return nil
```

```
}
```

네트워크 토폴로지 구성

노드 타입

1. 검증자 노드 (ModeValidator)

- 합의에 참여하는 노드
- 스테이킹된 토큰 보유
- 항상 온라인 상태 유지 필요

2. 풀 노드 (ModeFull)

- 일반적인 전체 노드
- 합의에 참여하지 않음
- 기본 프루닝 설정

3. 아카이브 노드 (ModeArchive)

- 모든 히스토리 데이터 저장
- 프루닝 없음
- 대용량 디스크 필요

4. 시드 노드 (ModeSeed)

- P2P 네트워크 부트스트래핑
- 장기간 실행되는 공개 주소
- 네트워크 크롤링 수행

장애 시뮬레이션 (Perturbations)

```
type Perturb string

const (
PerturbRestart Perturb = "restart"  // 컨테이너 재시작
PerturbStopStart Perturb = "stopstart"  // 정지 후 시작
PerturbRollback Perturb = "rollback"  // 롤백 후 재시작
PerturbUpgrade Perturb = "upgrade"  // 버전 업그레이드
// FuzzyHead 테스트
PerturbFuzzyHeadDropBlocks Perturb = "fuzzyhead_dropblocks"
PerturbFuzzyHeadDropMsgs Perturb = "fuzzyhead_dropmsgs"
PerturbFuzzyHeadAttRoot Perturb = "fuzzyhead_attroot"
PerturbFuzzyHeadMoreMsgs Perturb = "fuzzyhead_moremsgs"
)
```

검증자 업데이트

매니페스트에서 특정 높이에 검증자 세트 변경을 정의할 수 있습니다:

```
[validator_update.10]
fullnode01 = 100 # 높이 10에서 fullnode01을 100 스테이크로 검증자 추가
```

설정 파일 생성

Testnet 구조체

```go
type Testnet struct {
*e2e.Testnet
Manifest Manifest
Network netconf.ID
OmniEVMs []OmniEVM
AnvilChains []AnvilChain
PublicChains []PublicChain
SVMChains []SVMChain
Perturb  map[string][]Perturb
}
```

키 관리

각 노드와 서비스는 고유한 키 세트를 가집니다:

```go
type KeyType string

const (
KeyTypeConsensus KeyType = "consensus"  // 합의 참여용
KeyTypeValidator KeyType = "validator"  // 검증자 서명용
KeyTypeNetwork KeyType = "network"  // P2P 네트워킹용
KeyTypeEVM KeyType = "evm"  // EVM 트랜잭션용
```

```
)
```

네트워크 구성

```
func (m Manifest) EVMChains() ([]evmchain.Metadata, error) {
var  resp []evmchain.Metadata
// 퍼블릭 체인 추가
for  _, chain := range  m.PublicChains {
meta, ok := evmchain.MetadataByName(chain)
if !ok {
return  nil, errors.New("unknown public chain", "name", chain)
}
resp = append(resp, meta)
}
// Anvil 체인 추가
for  _, chain := range  m.AnvilChains {
meta, ok := evmchain.MetadataByName(chain)
if !ok {
return  nil, errors.New("unknown anvil chain", "name", chain)
}
resp = append(resp, meta)
}
// Omni 실행 체인 추가
```

```
meta, ok := evmchain.MetadataByID(m.Network.Static().OmniExecutionChainID)

if !ok {

return  nil, errors.New("unknown omni execution chain")

}

return  append(resp, meta), nil

}
```

정리

E2E 테스트 프레임워크의 매니페스트 및 네트워크 구성 시스템은:

1. **유연한 토폴로지 정의**: TOML 매니페스트를 통한 선언적 네트워크 구성
2. **다중 환경 지원**: Docker와 VM 기반 배포 모두 지원
3. **장애 시뮬레이션**: 다양한 perturbation을 통한 복원력 테스트
4. **확장 가능한 아키텍처**: 새로운 체인과 서비스 쉽게 추가 가능
5. **자동화된 설정**: 템플릿 기반 설정 파일 자동 생성

이 시스템을 통해 개발자들은 복잡한 멀티체인 환경에서 Omni 프로토콜의 모든 측면을 체계적으로 테스트할 수 있습니다.